



Drone cible d'entraînement pour la marine nationale



X



Sebastien GUIVARCH

Denis CAILLE

Francesa BREHONNET

Timéo BERNARD GOARANT

Gatien JAUMOUILLE

Nolan RIBETTE

1. Introduction	3
1.1 Contexte du projet.....	3
1.2 Objectif du projet.....	3
1.3 Intérêt du projet.....	4
2. Présentation des outils du projet.....	4
2.1 GitHub.....	4
2.2 Google Drive.....	5
2.3 Synthèse.....	5
3. Environnement de développement.....	6
3.1 Justification des outils et langages choisis	6
3.2 Justification des librairies utilisées.....	6
3.3 Contraintes du client.....	6
4. Architecture du projet.....	7
4.1 Diagramme d'architecture logicielle et matérielle	7
4.2 Schéma réseau commenté.....	8
5. Développement – état d'avancement.....	9
5.1 Présentation des composants développés par l'étudiant.....	9
5.2 Extraits de code commentés et expliqués.....	10
5.3 Tableau ou cahier de tests unitaires.....	12
6. Exploitation réseau et matériels	12
6.1 Présentation des équipements.....	12
6.2 Justification des choix matériels.....	14
6.3 Résultats d'un outil d'analyse réseau	15
6.4 Gestion des versions et cybersécurité.....	15



1.Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du BTS CIEL option A au lycée Vauban de Brest, en partenariat avec la Marine nationale et l'IRENAV (Institut de Recherche de l'École Navale).

Il répond à un besoin opérationnel réel de la Marine : entraîner les équipages à faire face à des menaces navales modernes.

Depuis plusieurs années, notamment en mer Rouge, des attaques sont menées contre des navires militaires ou civils à l'aide de petits navires rapides (skiffs) ou de drones de surface autonomes (aussi appelé USV : Unmanned Surface Vehicule). Ces menaces sont peu coûteuses, rapides, difficiles à détecter et représentent un risque important.

Dans ce contexte, la Marine nationale souhaite disposer de drones cibles réalistes, capables de simuler ce type d'attaque lors d'exercices. Le projet qui nous est confié consiste donc à concevoir un prototype de drone naval de surface, à faible coût, destiné à être utilisé lors d'entraînements.

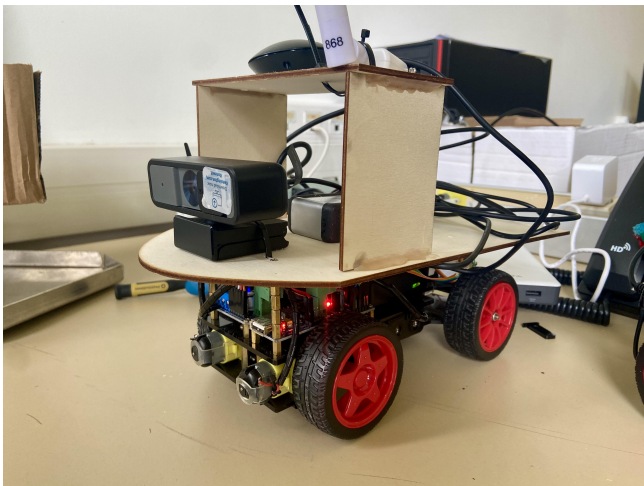
1.2 Objectif du projet

L'objectif principal du projet est de concevoir et réaliser un prototype fonctionnel de drone naval de type USV servant de cible d'entraînement pour la Marine nationale.

Plus précisément, le drone doit être capable de :

- Être piloté à distance par un opérateur (mode manuel),
- Suivre un itinéraire autonome à partir de points GPS (mode autonome),
- Se verrouiller sur une cible détectée par caméra et se diriger vers elle (mode verrouillage),
- Fonctionner avec des technologies peu coûteuses et des composants du commerce (approche DIY),
- Être suffisamment fiable pour des essais réels tout en restant économique, car il est conçu pour être potentiellement détruit lors des exercices.

Ce projet sert également de preuve de concept, afin d'évaluer la faisabilité de solutions low-cost pour des applications d'entraînement militaire.



1.3 Intérêt du projet

Ce projet présente un triple intérêt : opérationnel, technologique et pédagogique.

- Intérêt opérationnel
 - Il répond à un besoin concret de la Marine nationale en matière d'entraînement face à des menaces modernes.
 - Il permet de simuler des scénarios réalistes sans mobiliser des équipements coûteux.
 - Il contribue à améliorer la préparation et la réactivité des équipages.
- Intérêt technologique
 - Le projet mobilise des technologies actuelles : ROS 2, GPS, vision par caméra, communication LoRa, Docker.
 - Il explore l'utilisation de solutions low-cost dans un contexte militaire, ce qui est stratégique.
- Intérêt pédagogique
 - Il nous permet, étudiants, de travailler sur un projet réel et professionnalisant.
 - Il développe des compétences clés du BTS CIEL : conception système, développement logiciel, communication, tests, documentation et travail en équipe.
 - Il met les étudiants en situation proche de l'ingénierie industrielle et de la recherche appliquée.

2. Présentation des outils du projet

2.1 GitHub

GitHub est utilisé comme outil central de gestion du projet. Il permet de centraliser l'ensemble du code source, d'assurer le travail collaboratif entre les membres de l'équipe et de conserver l'historique des modifications ainsi qu'un suivi du projet. Grâce au système de versions, chaque évolution du projet est tracée et sécurisée, ce qui facilite la correction des erreurs et la validation des développements. GitHub est également utilisé pour héberger la documentation technique, garantissant ainsi la continuité, la fiabilité et la maintenabilité du projet.

Dans le GitHub, nous avons créé une arborescence simple et intuitive.

- SysML : contient les différents SysML du projet.
- Simulation : contient les scripts de tests et de simulation du projet.
- Docs technique : l'entièreté des documentation technique des membres du groupe.
- Code : vous y trouverez le code source du projet.

Les fonctionnalités de gestion de projet de GitHub ont été utilisées pour organiser et suivre l'avancement du travail. Les issues permettent de décrire les tâches à réaliser, les anomalies à corriger ou les améliorations à apporter, tout en les attribuant à un membre de l'équipe. Le tableau Kanban offre une vision claire de l'état du projet en classant les tâches selon leur progression (à faire, en cours, terminé). Le travail est découpé en sprints, correspondant à des périodes de développement définies, durant lesquelles des objectifs précis sont fixés. Cette organisation permet un suivi rigoureux, une meilleure répartition des tâches et une adaptation rapide en cas de difficulté.

2.2 Google Drive

Google Drive est utilisé comme espace de stockage et de partage des documents du projet. Il permet de centraliser les fichiers non liés au code, tels que les documents de conception, les comptes rendus de réunion, et les schémas. Cet outil facilite le travail collaboratif en assurant un accès commun aux informations à jour, tout en évitant les doublons et les pertes de données. Google Drive contribue ainsi à une organisation efficace et à la continuité du projet.

Voici son arborescence :

- Compte rendu : contient les comptes rendu des différents sprints.
- Schémas : les différents schémas en lien avec notre projet.
- Soutenance : contient des documents utiles pour la rédaction de la soutenance, le rapport commun ainsi que 3 dossiers :
 - Timéo
 - Gatien
 - Nolan

Ils contiennent les rédactions de soutenance de chaque membre du groupe.

Google Drive nous sert bien uniquement à déposer des documentations ainsi que des schéma. Vous n'y trouverez en aucun cas du codes dedans.

2.3 Synthèse

Cette organisation a été choisie afin de séparer clairement le développement logiciel de la gestion documentaire. GitHub est dédié au code source, au suivi des tâches et à la collaboration technique, tandis que Google Drive est utilisé pour le partage et la centralisation des documents du projet. Cette complémentarité facilite le travail en équipe en offrant à chacun un accès rapide aux informations nécessaires, réduit les risques d'erreurs ou de doublons et améliore la coordination entre les membres du groupe. Elle permet ainsi un suivi structuré, efficace et collaboratif du projet.

3. Environnement de développement

3.1 Justification des outils et langages choisis

Dans le cadre de ce projet, le choix des outils a été guidé par des contraintes de performance, de compatibilité embarquée et de facilité de développement.

 Le langage Python 3 a été retenu car il permet un développement rapide et une grande lisibilité du code. Il est particulièrement adapté à ROS2 grâce à la bibliothèque `rclpy`, qui facilite la création de nœuds et la gestion des communications. De plus, Python est largement utilisé dans les domaines de la robotique et de l'intelligence artificielle.

Le framework ROS2 constitue le cœur de l'architecture logicielle. Il permet de structurer le système sous forme de nœuds indépendants communiquant entre eux via des *topics*. Ce modèle favorise la modularité, la maintenabilité et la réutilisation du code. ROS2 est également conçu pour des systèmes distribués et temps réel, ce qui correspond aux besoins du projet.

L'utilisation de Docker permet de garantir un environnement de développement identique entre les différentes machines (poste de développement et Raspberry Pi). Cela évite les problèmes de dépendances et simplifie le déploiement du projet. De plus `ros2` est compatible sur docker.

Enfin, GitHub est utilisé pour le versionnement du code. Il permet de suivre l'évolution du projet, de conserver un historique des modifications et de structurer le travail en différentes versions.

3.2 Justification des librairies utilisées

Plusieurs librairies ont été intégrées afin de répondre aux besoins spécifiques du projet.

La bibliothèque `rclpy` permet de créer et gérer les nœuds ROS2 en Python. Les packages `std_msgs` et `sensor_msgs` sont utilisés pour définir les formats de communication entre les différents composants.

 Pour la partie vision, la bibliothèque OpenCV (`cv2`) est utilisée pour traiter les images issues de la caméra. Elle permet de manipuler les flux vidéo en temps réel.

Le framework PyTorch (`torch`) est utilisé pour exécuter le modèle de détection YOLOv5, permettant d'identifier des objets (bateaux) dans les images.

Enfin, `cv_bridge` assure la conversion entre les formats d'images ROS et OpenCV, ce qui est indispensable pour intégrer la vision dans ROS2.

3.3 Contraintes du client

Le projet doit répondre à plusieurs contraintes.

Sur le plan matériel, le système repose sur un Raspberry Pi 5, qui possède des ressources limitées en calcul et en mémoire. Il est donc nécessaire d'optimiser les traitements, notamment pour la vision par ordinateur.

Sur le plan fonctionnel, le robot doit être capable de réagir rapidement à son environnement, ce qui impose des contraintes de temps réel.

Enfin, des contraintes de sécurité sont présentes : le robot doit être capable de s'arrêter automatiquement en cas de perte de communication ou de défaillance d'un composant. Cela justifie la mise en place d'un système de watchdog.

4. Architecture du projet

4.1 Diagramme d'architecture logicielle et matérielle

L'architecture du projet repose sur une organisation modulaire en plusieurs nœuds ROS2, chacun ayant un rôle spécifique.

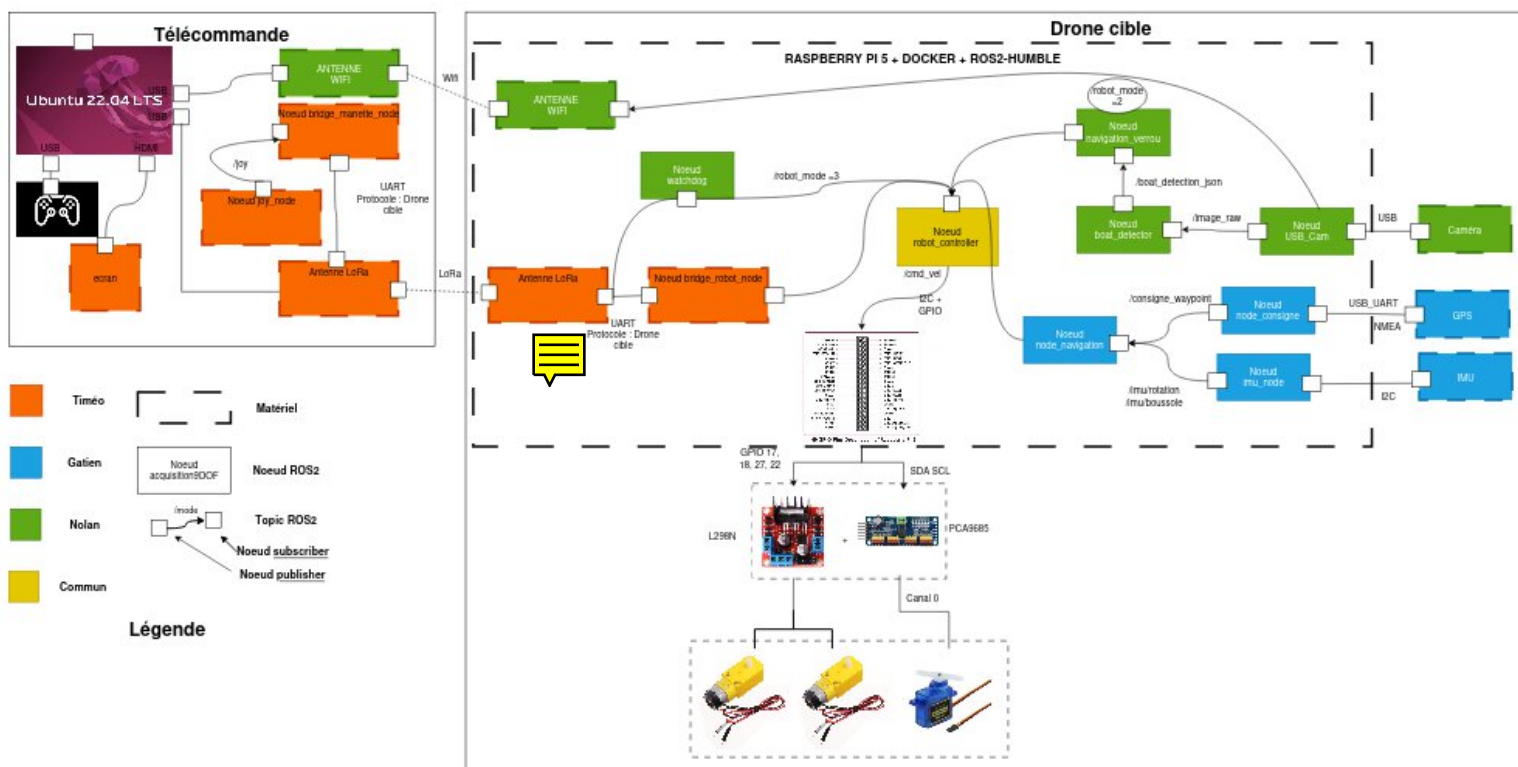


Schéma 1 : Diagramme d'architecture logicielle et matérielle

Le nœud USB_cam récupère les images de la caméra pour les publier sur le topic /image_raw. Les informations sont ensuite transmises au nœud boat_detector ainsi qu'au pc poste pilotage.

Le nœud Boat Detector récupère les images de la caméra et détecte les objets à l'aide de YOLOv5. Les informations sont ensuite transmises au nœud de Navigation avec le topic /boat_detection_json qui fait apparaître les coordonnées du rectangle où a eu lieu la détection puis la hauteur et largeur du rectangle.

Le nœud Navigation_verrou permet de faire fonctionner les moteurs en fonction de si il y a eu une détection d'un bateau et de le suivre. Ce nœud s'active quand /robot_mode passe à 2 ce qui signifie que le mode verrouillage est activé.

Le nœud Robot Controller permet de contrôler les moteurs avec le topic /cmd_vel, ce topic publie les données des axes X, Y, Z. Ce nœud s'adapte donc au commande envoyés puis fait fonctionner les moteurs en fonction des requêtes.

Le nœud Watchdog surveille le neoud bridge opérateur qui fait la liaison entre le poste pilotage et le drone depuis l'antenne radio Lora. En cas de problème le watchdog passe le drone en mode 3 = safemode, ce qui signifie que tout les moteurs du robot s'arrête.

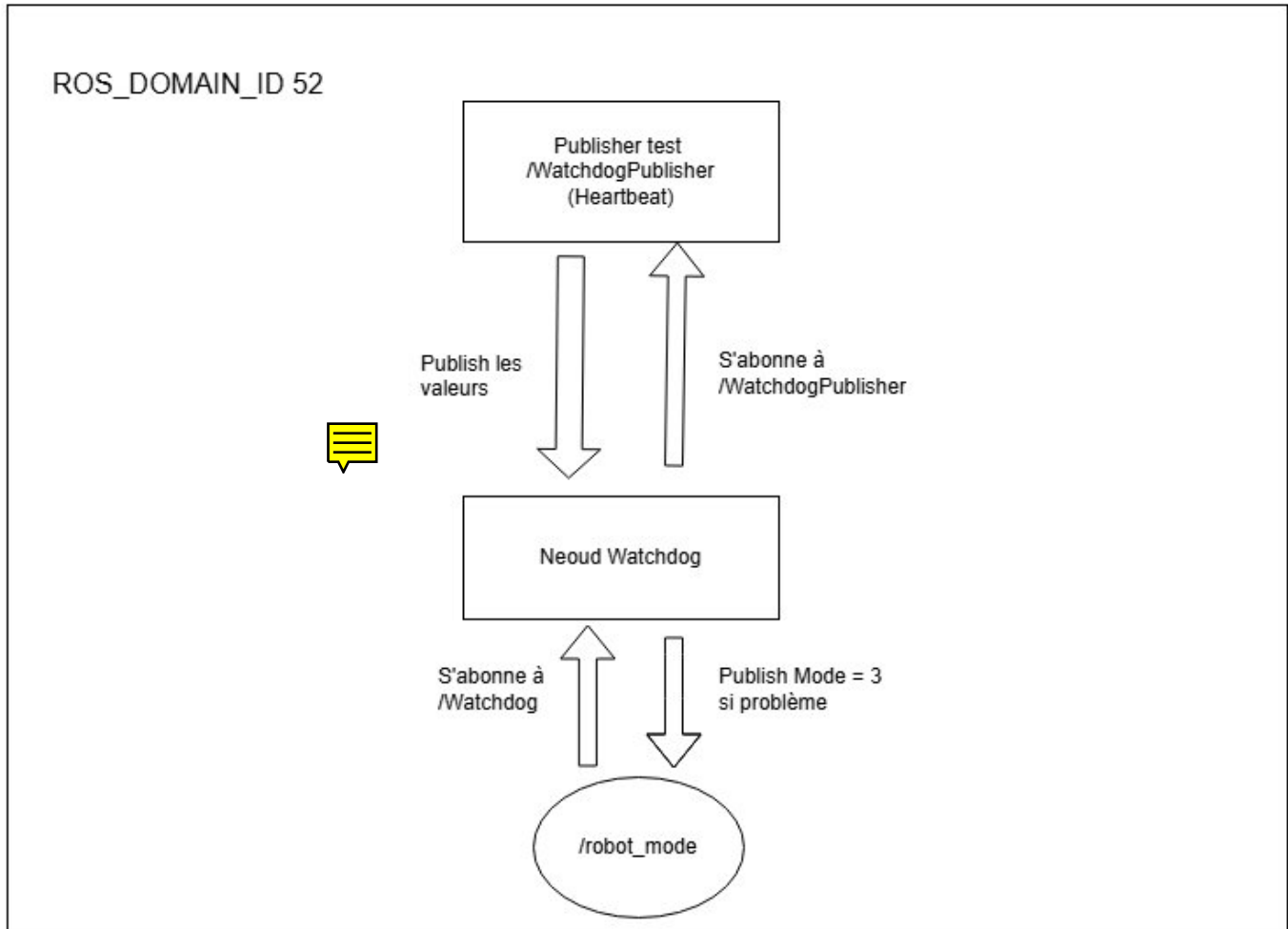


Schéma 2 : nœud Watchdog

4.2 Schéma réseau commenté

Le système est composé de deux entités principales : un poste de développement et un robot embarqué.

PC (développement) --- SSH ---> Raspberry Pi (ROS2 + Docker)

Le Raspberry Pi exécute les nœuds ROS2 à l'intérieur d'un conteneur Docker. Le poste de développement permet de déployer le code, surveiller les logs et interagir avec le système via des commandes ROS2.

Les communications internes entre nœuds utilisent le protocole DDS, intégré à ROS2, qui permet des échanges rapides et fiables. (je précise que pour le moment je surveille Publisher test et non bridge_operator pour des raisons en d'attente du camarade qui le développe).

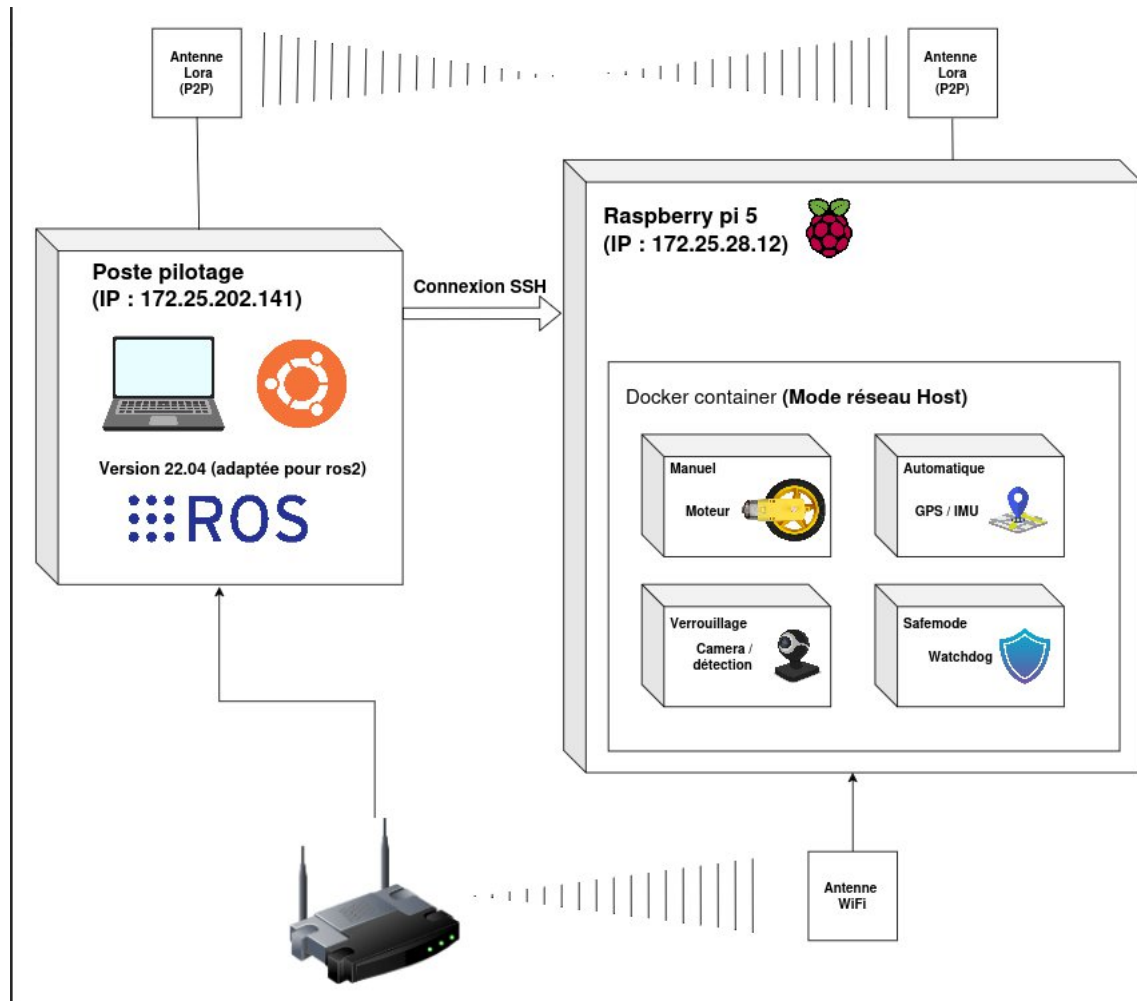


Schéma 3 : Schéma réseau en mode maintenance / développement

5. Développement – état d'avancement

5.1 Présentation des composants développés par l'étudiant

Plusieurs composants ont été développés dans ce projet.

Le nœud pour récupérer les images de la caméra pour ensuite les distribuer aux différents services.

Le nœud de détection permet d'analyser les images et d'identifier des objets. Il constitue une brique essentielle pour la perception du robot.

Le nœud de navigation en mode verrouillage assure l'interface avec le matériel en pilotant les moteurs.

Enfin, le nœud watchdog joue un rôle critique en surveillant le bon fonctionnement du système. Il permet de garantir la sécurité en cas de défaillance.

5.2 Extraits de code commentés et expliqués

Voci un extrait du code qui illustre le fonctionnement du watchdog :

```
def check_watchdog(self):
    """
    Vérifie périodiquement l'état de santé de la communication.
    Si aucun message n'est reçu pendant plus de 3 secondes,
    une procédure de mise en sécurité est déclenchée.
    """
    # Calcul du temps écoulé depuis la dernière réception
    delta = (datetime.now() - self.last_msg_time).total_seconds()

    if delta > 3.0:
        self.get_logger().warn(f"Alerte : Perte de signal ({delta:.2f}s) -> Passage en mode
Sécurité")

        # Création du message d'arrêt d'urgence
        msg = String()
        msg.data = "3" # Le mode "3" correspond à l'arrêt complet des moteurs

        # Publication de l'ordre de coupure vers les actionneurs
        self.mode_publisher.publish(msg)

        # Réinitialisation pour éviter une boucle d'alerte infinie
        self.last_msg_time = datetime.now()
```

Afin de prévenir tout comportement erratique du robot en cas de défaillance logicielle ou de perte de connexion réseau, nous avons implémenté un nœud de surveillance dédié.

Le principe est simple : un "battement de cœur" (heartbeat) est attendu sur le topic /WatchdogPublisher. Si le nœud WatchdogMonitorNode ne détecte aucune activité pendant un seuil critique de 3 secondes, il considère que le système est instable. Il publie alors immédiatement la valeur "3" sur le topic /robot_mode, ce qui force l'arrêt des moteurs et le passage du robot en mode de sécurité.

Voci un autre extrait de code qui illustre le fonctionnement du nœud navigation en mode verrouillage :

```
def control_loop(self):
    """
    Boucle de contrôle (20 Hz) : Calcule les commandes moteurs
    pour centrer le robot sur le bateau détecté.
    """
```

```
twist = Twist()

# Sécurité : Si le mode "verrou" (2) n'est pas actif, on stoppe tout
if not self.locked or not self.last_detection:
    self.cmd_pub.publish(twist)
    return

# Extraction des coordonnées du premier bateau détecté par YOLOv5
boat = self.last_detection["boats"][0]
center_x = float(boat.get("x_min", 0.0)) + (float(boat.get("width", 0.0)) / 2.0)

# Calcul de l'erreur de centrage (différence entre le centre du bateau et le centre de
l'image)
error = center_x - self.image_center_x

# Paramètres du correcteur Proportionnel (P)
Kp = 0.002
dead_zone = 20.0 # Zone morte en pixels pour éviter les oscillations

# Commande de poussée constante
twist.linear.x = 0.6

# Calcul de la vitesse de rotation (Angulaire Z)
if abs(error) > dead_zone:
    # Plus l'erreur est grande, plus le robot tourne vite pour compenser
    twist.angular.z = float(-Kp * error)

# Envoi de la commande de mouvement au topic /cmd_vel
self.cmd_pub.publish(twist)
```

Le nœud `NavigationNode` assure le suivi automatique d'une cible (ici, un bateau) détectée par l'algorithme YOLOv5. Le fonctionnement repose sur un correcteur proportionnel (P) simple mais efficace :

Récupération des données : Le code parse un message JSON contenant les coordonnées `x_min` et la largeur de la "Bounding Box" fournie par l'IA.

Calcul de l'erreur : On compare le centre horizontal du bateau avec le centre théorique de la caméra (320 px).

Loi de commande : > * Une vitesse linéaire constante (0.6 m/s) est appliquée pour avancer vers la cible.

Une vitesse angulaire est calculée en multipliant l'erreur par un gain `Kp`. Si le bateau est à droite, le robot pivote à droite, et inversement.

Zone morte (Dead Zone) : Une tolérance de 20 pixels est instaurée pour éviter que les moteurs ne forcent inutilement lorsque le bateau est déjà quasiment centré, limitant ainsi les oscillations de trajectoire.⁷

5.3 Tableau ou cahier de tests unitaires

Des tests ont été réalisés pour valider le bon fonctionnement des composants.

Un test du watchdog a permis de vérifier que le robot passe en mode safemode après 3 secondes sans message en regardant le topic `/robot_mode`. Pour simuler le nœud `bridge_operator` qui fait la liaison entre le drone et le poste opérateur j'ai développé un script python se nommant `PublisherWatchdog` qui publie des valeurs toutes les secondes en partant de 1000 pour faire une simulation de crash à la valeur 1050 qui fait donc réagir le watchdog.

Enfin, des tests sur la détection ont été réalisés, bien que des améliorations soient encore possibles sur la précision du modèle.

(À faire)

6. Exploitation réseau et matériels

6.1 Présentation des équipements

Sur ma partie le système repose sur plusieurs équipements :

- une Raspberry Pi 5
- une webcam
- 2 antennes Wifi (réceptrice, émettrice)
- 2 moteurs
- 1 servomoteur
- 1 PCA9685
- 1 L298N

1. L'Unité Centrale : Le Cerveau du Drone

Raspberry Pi 5

C'est le cœur du système embarqué. Sa puissance de calcul accrue par rapport aux versions précédentes est indispensable ici pour :

- Faire tourner ROS 2 de manière fluide.
- Exécuter l'algorithme de détection d'objets YOLOv5 en temps réel sur le flux vidéo.
- Gérer les communications réseau et la logique de navigation (le script `NavigationNode`).

2. Perception et Communication : Les Sens

Webcam (USB)

Elle sert d'œil au robot. Elle capture le flux vidéo de l'environnement marin. Ce flux est ensuite découpé en images (frames) qui sont analysées par le modèle YOLOv5 pour repérer si il y a des cibles ou non.

Antennes Wifi (Émettrice / Réceptrice)

Elles assurent la liaison de données à longue portée.

- Utilité : Permet le retour vidéo en direct vers le poste opérateur sur un autre bateau.
- Fiabilité : L'utilisation d'antennes à gain permet de conserver l'affichage de la caméra du drone même lorsqu'il s'éloigne des côtes.

3. Électronique de Puissance et Contrôle : Le Système Nerveux

PCA9685 (Contrôleur PWM via I2C)

Comme la Raspberry Pi possède un nombre limité de sorties PWM précises, on utilise ce module.

- Rôle : Il reçoit des ordres de la Raspberry Pi via le protocole I2C et génère des signaux PWM (Pulse Width Modulation) très stables.
- Utilité : Il pilote à la fois le servomoteur de direction et le contrôleur de vitesse des moteurs, libérant ainsi les ressources CPU de la Raspberry.

L298N (Pont en H / Driver de moteur)

C'est l'interface de puissance entre la batterie et les moteurs.

- Rôle : Il permet de contrôler le sens de rotation et la vitesse des deux moteurs à courant continu.
- Sécurité : Il isole la partie commande (basse puissance) de la partie puissance pour protéger la Raspberry Pi des pics de tension.

4. Actionneurs : Les Muscles

2 Moteurs à courant continu (DC)

Ils assurent la propulsion du drone. Dans le code, ils reçoivent les commandes `linear.x` pour faire avancer le bateau vers la cible détectée.

1 Servomoteur

Il est utilisé pour le gouvernail. Il permet d'orienter le flux d'eau ou de faire pivoter une hélice pour diriger le drone. C'est lui qui réagit à la commande `angular.z` calculée par mon script de navigation.

6.2 Justification des choix matériels

1. Puissance de Calcul et Vision par Ordinateur

Pourquoi la Raspberry Pi 5 ?

Le choix de la Raspberry Pi 5 s'est imposé par la nécessité d'exécuter des modèles de deep learning (YOLOv5) tout en gérant l'écosystème ROS 2.

- Performances CPU/GPU : Contrairement à la version 4, la Pi 5 offre une bande passante et une vitesse de traitement permettant de traiter le flux vidéo de la webcam avec une latence minimale, ce qui est vital pour un asservissement en temps réel.
- Écosystème ROS 2 : C'est la plateforme de référence pour le prototypage robotique, offrant une compatibilité totale avec les bibliothèques de communication (rclpy) et de traitement d'image (OpenCV).

2. Précision du Contrôle et Gestion de l'Énergie

Le couple PCA9685 & L298N

Plutôt que de brancher les moteurs directement sur la Raspberry Pi, nous avons opté pour une architecture séparée :

- Déportation du PWM (PCA9685) : Ce module communique en I2C, n'utilisant que deux broches sur la Raspberry Pi. Il permet de générer des signaux PWM d'une précision de 12 bits, indispensable pour un angle de gouvernail (servomoteur) précis et une vitesse moteur fluide.
- Puissance et Isolation (L298N) : Le pont en H L298N permet de gérer des courants élevés pour les deux moteurs de propulsion tout en protégeant la Raspberry Pi des retours de courant (Back-EMF) grâce à son isolation électrique.

3. Communication et Portée

Antennes Wi-Fi Longue Portée

En milieu aquatique, la portée du Wi-Fi standard d'un ordinateur est insuffisante.

- Liaison de données robuste : L'utilisation d'antennes externes (émettrice sur le drone, réceptrice sur la station au sol) permet de maintenir le lien pour le Watchdog. Si la connexion tombe, le script de sécurité coupe les moteurs.
- Monitoring vidéo : Le débit élevé du Wi-Fi est nécessaire pour streamer le flux de la webcam, permettant un contrôle à distance sécurisé.

4. Architecture de la Propulsion

2 Moteurs DC et 1 Servomoteur

Cette configuration a été choisie pour sa simplicité et sa réactivité :

- Propulsion : Les deux moteurs fournissent la poussée nécessaire pour contrer les courants marins.

- Direction : Le servomoteur offre une direction plus précise qu'une direction différentielle (variation de vitesse entre deux moteurs), ce qui est préférable pour la stabilité d'un drone bateau à haute vitesse.

6.3 Résultats d'un outil d'analyse réseau

6.4 Gestion des versions et cybersécurité

Le projet utilise GitHub pour assurer la gestion des versions du code. Cet outil permet de suivre précisément les modifications apportées au projet grâce à un historique des commits. Il facilite également le retour à une version antérieure en cas de problème et permet une meilleure organisation du développement via l'utilisation de branches. Cette approche garantit une traçabilité complète et améliore la qualité du projet.

Sur le plan de la cybersécurité, plusieurs mesures concrètes ont été mises en œuvre afin de sécuriser le système.

Tout d'abord, l'utilisation de Docker permet d'isoler l'environnement d'exécution. Les nœuds ROS2 sont exécutés dans des conteneurs, ce qui limite les interactions avec le système principal et réduit les risques liés à une mauvaise configuration ou à une attaque.

Ensuite, la connexion au Raspberry Pi se fait via le protocole SSH sécurisé par clé. Une paire de clés SSH (clé publique / clé privée) a été générée. La clé publique a été ajoutée sur le Raspberry Pi dans le fichier `authorized_keys`, tandis que la clé privée est conservée sur le poste de développement.

Cette méthode présente plusieurs avantages importants :

- elle évite l'utilisation de mots de passe, qui peuvent être interceptés ou devinés,
- elle renforce l'authentification grâce à un système cryptographique,
- elle protège contre les attaques par brute force,
- elle permet d'automatiser les connexions pour le déploiement ou les scripts.

De plus, des améliorations de sécurité peuvent être appliquées au service SSH, comme :

- la désactivation de l'authentification par mot de passe,
- la restriction des utilisateurs autorisés à se connecter,
- la modification du port SSH par défaut.

Enfin, un mécanisme de sécurité fonctionnelle est assuré par le watchdog. Ce composant surveille l'activité du robot et détecte toute perte de communication. En cas de problème (absence de message pendant un certain temps), il déclenche automatiquement un arrêt du robot en envoyant une commande de sécurité.